



SGCI

Sistema de Gestão de Crises Institucionais

Documentação Técnica — Arquitetura e Código

Quarkus 3.37 · Java 21

Angular 20.3

PostgreSQL + Flyway

JWT via Cookie

Gerado em 06/07/2026

Sumário

1. Visão geral do sistema
2. Arquitetura e stack tecnológica
3. Estrutura de diretórios
4. Modelo de domínio (entidades e enums)
5. Banco de dados e migrações Flyway
6. Backend — camadas (Controller → Service → Repository)
7. Backend — mapa completo de endpoints REST
8. Segurança, autenticação e autoria (JWT + auditoria)
9. Regras de negócio principais
10. Frontend — arquitetura Angular
11. Frontend — rotas, guards e interceptors
12. Frontend — funcionalidades (features)
13. Fluxos de ponta a ponta
14. Como executar o projeto

1. Visão geral do sistema

O **SGCI (Sistema de Gestão de Crises Institucionais)** é uma aplicação web full-stack para registrar, acompanhar e encerrar crises em uma instituição de ensino (campi, departamentos), documentar as ações tomadas durante cada crise e emitir relatórios finais, com trilha de auditoria de todas as alterações.

O sistema é dividido em dois projetos independentes que se comunicam por HTTP/JSON:

BACKEND — API REST

Quarkus (Java 21). Expõe endpoints REST, aplica regras de negócio, persiste em PostgreSQL e cuida da autenticação por JWT em cookie HttpOnly.

```
groupId: br.com.crisismanagement  
artifactId: crisis-management-api
```

FRONTEND — SPA

Angular 20 (100% standalone, com signals). Renderiza as telas, valida sessão a cada navegação e consome a API. Servido em dev por `ng serve` na porta 4200.

Conceitos do domínio

Conceito	Descrição
Crise	Ocorrência central. Tem título, descrição, nível de criticidade, status (ciclo de vida) e vínculos a tipo, campus, cenário e departamento.
Ação de crise	Registro cronológico do que foi feito durante a crise (contenção, comunicação, monitoramento, resolução).
Relatório de crise	Documento final único por crise, permitido só após a crise estar resolvida/encerrada.
Dados de referência	Instituições, Campi, Departamentos, Tipos de Crise e Cenários — cadastros de apoio usados nos formulários.
Auditoria	Log imutável de toda operação de modificação (quem, o quê, quando, resultado HTTP).
Usuário / Perfil	Contas com perfil <code>ADMIN</code> (acesso total) ou <code>COMUM</code> (acesso restrito ao próprio departamento).

2. Arquitetura e stack tecnológica

Padrão arquitetural

O backend segue uma **arquitetura em camadas** clássica, com nomes que o próprio código documenta como *BO* (*Business Object*) para os services e *DAO* para os repositories:

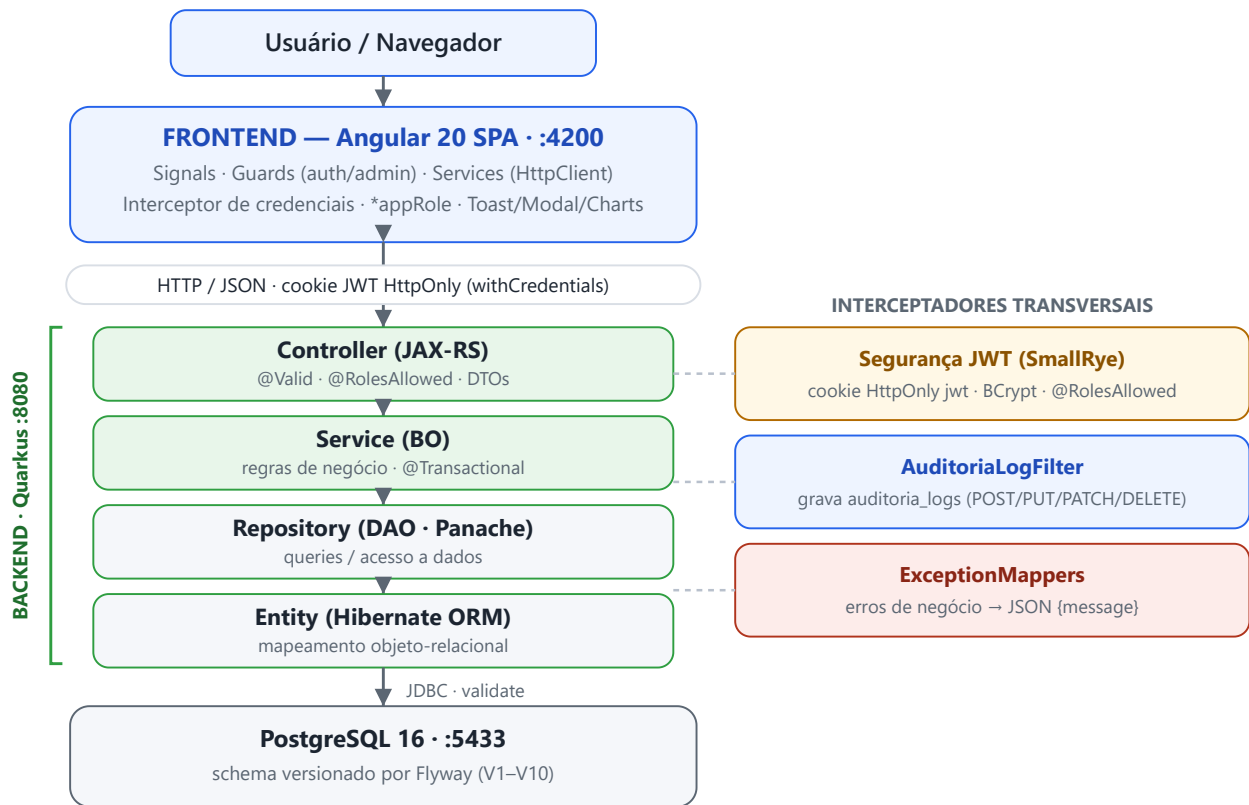


Figura 1 — Fluxo do sistema: da requisição no navegador até o banco, atravessando as camadas do backend e os interceptadores transversais.

A comunicação com o exterior usa **DTOs** (records Java): `*Request` para entrada (com Bean Validation) e `*Response` para saída — as entidades JPA nunca são expostas diretamente.

Stack do backend

Tecnologia	Versão	Papel
Java	21	Linguagem (records, switch expressions, etc.)
Quarkus	3.37.1	Framework (BOM <code>quarkus-bom</code>)
quarkus-rest + rest-jackson	—	Endpoints REST (JAX-RS) e serialização JSON
Hibernate ORM + Panache	—	ORM com repositórios ativos
quarkus-jdbc-postgresql	—	Driver do banco
Flyway	—	Versionamento/migração de schema
SmallRye JWT (+ build)	—	Emissão e verificação de JWT
Elytron Security Common	—	<code>BcryptUtil</code> para hash de senha
Hibernate Validator	—	Bean Validation nos DTOs
JUnit 5 + REST-assured	—	Testes (escopo de teste)

Stack do frontend

Tecnologia	Versão	Papel
Angular	20.3	Framework SPA, 100% standalone (sem NgModules)
TypeScript	5.9	Linguagem
RxJS	7.8	Observables (HTTP, streams de navegação)
@angular/build	20.3	Novo application builder
SCSS	—	Design system próprio (sem Material/Bootstrap)
Signals	—	Estado reativo (<code>signal</code> , <code>computed</code> , <code>effect</code>)
Karma + Jasmine	—	Testes

Padrões modernos usados em todo o front: `inject()` (sem injeção por construtor), control-flow `@if` / `@for` , `LoadComponent` (lazy loading), `takeUntilDestroyed()` e diretiva estrutural com `input.required()` .

3. Estrutura de diretórios

BACKEND (SRC/MAIN)

```
br/com/crisismanagement/
├─ config/
│   └─ DataSeeder.java          (usuários iniciais)
├─ controllers/                 (11 controllers REST)
├─ dto/                         (Request/Response records)
├─ entities/
│   └─ enums/                   (4 enums de domínio)
│       └─ *.java               (9 entidades)
├─ exceptions/                 (exceção + mapper)
├─ repositories/               (10 repositórios Panache)
├─ security/
│   └─ AuthCookieFactory
│       └─ AuditoriaLogFilter
└─ services/                   (11 services / BOs)

resources/
├─ application.properties
├─ db/migration/ V1 ... V8      (Flyway)
├─ privateKey.pem / publicKey.pem
└─ docker/ Dockerfile.*
```

FRONTEND (FRONTEND/SRC)

```
app/
├─ app.ts / app.config.ts / app.routes.ts
├─ core/
│   └─ config/api.config.ts
│   └─ guards/ auth · admin
│   └─ interceptors/ credentials
│   └─ directives/ role
│   └─ models/ auth · user
│       └─ services/ auth.service
├─ layouts/
│   └─ main-layout/             (casca autenticada)
└─ features/
    └─ auth/                    (login)
        └─ dashboard/
            └─ crises/           (list · form · detail)
                └─ relatorios/   (list · form)
                    └─ auditoria/ (list, admin)
                        └─ departamentos/ tipos-crise/
                            └─ instituicoes/ campi/ cenarios/
                                (cada feature: models · pages · services)
```

4. Modelo de domínio (entidades e enums)

Todas as entidades estendem `PanacheEntityBase` com campos públicos (padrão Panache) e id `@GeneratedValue(IDENTITY)` . Os relacionamentos são modelados por **chaves estrangeiras como Long** (ex.: `tipoCriseId`), e não por associações JPA `@ManyToOne` — uma escolha de design que mantém as entidades simples e o controle das joins no serviço.

Enums

Enum	Valores	Uso
<code>NivelCriticidade</code>	BAIXO · MEDIO · ALTO · CRITICO	Gravidade da crise
<code>StatusCrise</code>	ABERTA · EM_ANDAMENTO · RESOLVIDA · ENCERRADA	Ciclo de vida da crise
<code>TipoAcao</code>	CONTENCAO · COMUNICACAO · MONITORAMENTO · RESOLUCAO	Categoria da ação registrada
<code>PerfilUsuario</code>	ADMIN · COMUM	Papel de acesso — o nome vira a <i>role</i> no claim <code>groups</code> do JWT

Entidades

Entidade / Tabela	Campos principais
User <code>users</code>	id, name, email (único), password (bcrypt), perfil (enum), departamentold, active, createdAt
Crise <code>crises</code>	id, titulo, descricao, status , nivel , tipoCriseld, campusId, cenariold (opcional), departamentold, active, createdAt
AcaoCrise <code>acoes_crise</code>	id, descricao, tipo (TipoAcao), criseld, usuariold, createdAt
RelatorioCrise <code>relatorios_crise</code>	id, titulo, resumoOcorrido, impactosIdentificados, acoesRealizadas, resultadoFinal, recomendacoes, criseld (único), usuariold, createdAt
Instituicao <code>instituicoes</code>	id, nome, sigla, active, createdAt
Campus <code>campi</code>	id, nome, instituicaold (FK), active, createdAt
Departamento <code>departamentos</code>	id, nome, sigla, active, createdAt
TipoCrise <code>tipos_crise</code>	id, nome, descricao, active, createdAt
CenarioCrise <code>cenarios_crise</code>	id, descricao, nivelSugerido, tipoCriseld (FK), departamentold (FK), active, createdAt
AuditoriaLog <code>auditoria_logs</code>	id, usuariold, usuarioNome, acao, metodoHttp, entidadeAfetada, recurso, statusResposta, dataHora

EXEMPLO DE ENTIDADE — `CRISE.JAVA`

```
@Entity @Table(name = "crises")
public class Crise extends PanacheEntityBase {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;
    @Column(nullable = false, length = 200) public String titulo;
    @Column(nullable = false, length = 1000) public String descricao;
    @Enumerated(EnumType.STRING) public StatusCrise status;
    @Enumerated(EnumType.STRING) public NivelCriticidade nivel;
    public Long tipoCriseId;    public Long campusId;
    public Long cenarioId;     public Long departamentoId;    // FKs como Long
    public boolean active;     public LocalDateTime createdAt;
}
```


5. Banco de dados e migrações Flyway

O schema é gerenciado 100% por Flyway (`quarkus.flyway.migrate-at-start=true`); o Hibernate roda em modo `validate` (não altera tabelas, só confere que batem com as entidades). As migrações rodam em ordem na inicialização.

Migração	O que cria / altera
V1	Tabela users (id, name, email único, password, active, created_at).
V2	Tabelas departamentos e tipos_crise . Semeia 1 departamento ("Coordenação de TI"/CTI) e 1 tipo ("Tecnologia").
V3	Tabelas instituicoes , campi (FK → instituicoes) e cenarios_crise (FKs → tipos_crise, departamentos).
V4	Tabela crises com FKs para tipos_crise, campi, cenarios_crise (nullable) e departamentos.
V5	Tabela acoes_crise (FKs → crises, users).
V6	Tabela relatorios_crise — <code>crise_id</code> com constraint UNIQUE (1 relatório por crise).
V7	Adiciona colunas perfil (default 'COMUM') e departamento_id em users; promove <code>admin@crisis.local</code> a ADMIN.
V8	Tabela auditoria_logs + índices por <code>data_hora</code> DESC e por <code>entidade_afetada</code> .

Seed de aplicação (DataSeeder): na inicialização cria dois usuários se não existirem — `admin@crisis.local` / `Admin@123` (perfil ADMIN) e `usuario@crisis.local` / `Comum@123` (perfil COMUM, vinculado ao 1º departamento). As senhas são gravadas com hash BCrypt.

6. Backend — camadas (Controller → Service → Repository)

Controllers (11)

Recebem HTTP, validam o DTO com `@Valid`, aplicam a segurança declarativa (`@Authenticated` / `@RolesAllowed`) e delegam ao service. Padrão base do resource de crises:

```
@Path("/api/crises") @Authenticated
@Consumes(APPLICATION_JSON) @Produces(APPLICATION_JSON)
public class CriseController {
    @Inject CriseService criseService;

    @GET public List<CriseResponse> listAll() { ... }
    @POST @RolesAllowed("ADMIN")
    public Response create(@Valid CriseRequest request) { ... }
    @PATCH @Path("/{id}/status") @RolesAllowed("ADMIN")
    public CriseResponse changeStatus(@PathParam("id") Long id, @Valid StatusUpdateRequest r) { ... }
}
```

Services / BOs (11)

Service	Responsabilidades-chave
AuthService	Valida credenciais (BCrypt), emite JWT assinado, expõe usuário atual (<code>/me</code>).
CriseService	CRUD de crises, transição de status (bloqueia crise ENCERRADA), listagem/registro de ações com regra de departamento.
RelatorioCriseService	Cria relatório só para crise RESOLVIDA/ENCERRADA e apenas 1 por crise; grava autor a partir do JWT.
UsuarioService	CRUD administrativo: unicidade de e-mail, hash de senha, senha opcional na edição, impede auto-desativação.
AuditoriaService	Registra logs (transação <code>REQUIRES_NEW</code>), busca com filtros e monta <i>facets</i> (valores distintos).
DashboardService	Agrega contadores de crises por status e nível crítico.
Campus/Departamento/Instituicao/TipoCrise/CenarioCrise Service	CRUD dos dados de referência com <i>soft-delete</i> (desativa em vez de apagar).

Repositories / DAOs (10)

Implementam `PanacheRepository<T>`. Exemplos de queries customizadas:

Repository	Métodos além do CRUD
CriseRepository	listAllOrdered() (order by createdAt desc), countByStatus() , countByNivel()
UserRepository	findByEmail() , listAllOrdered()
RelatorioCriseRepository	existsByCrise() , listAllOrdered()
AcaoCriseRepository	listByCrise()
AuditoriaRepository	search(entidade, acao, usuario) (filtros dinâmicos), distinctEntidades/Acoes/Usuarios()

DTOs

Records por par Request / Response . Os Request carregam Bean Validation; os Response têm um método fábrica `from(entidade)` . Exemplo:

```
public record CriseRequest(
    @NotBlank @Size(max=200) String titulo,
    @NotBlank @Size(max=1000) String descricao,
    @NotNull NivelCriticidade nivel,
    @NotNull Long tipoCriseId, @NotNull Long campusId,
    Long cenarioId,           // opcional
    @NotNull Long departamentoId) {}
```

Categorias de DTO presentes: Auth (Login/User), Crise, AcaoCrise, RelatorioCrise, Usuario, Campus, Departamento, Instituicao, TipoCrise, CenarioCrise, Dashboard, Auditoria (Log + Facets) e `StatusUpdateRequest` .

7. Backend — mapa completo de endpoints REST

Legenda de acesso: **PÚBLICO** **AUTENTICADO** **ADMIN+COMUM** **SÓ ADMIN**

Método	Rota	Acesso	Função
POST	/auth/login	PÚBLICO	Autentica e devolve cookie <code>jwt</code>
POST	/auth/logout	AUTH	Expira o cookie
GET	/auth/me	AUTH	Usuário da sessão
GET	/api/dashboard/resumo	AUTH	Indicadores agregados
GET	/api/crises	AUTH	Lista crises
GET	/api/crises/{id}	AUTH	Detalha crise
POST	/api/crises	ADMIN	Cria crise
PATCH	/api/crises/{id}/status	ADMIN	Avança status
GET	/api/crises/{id}/acoes	AUTH	Lista ações da crise
POST	/api/crises/{id}/acoes	ADMIN+COMUM	Registra ação (COMUM só no seu depto.)
GET	/api/relatorios	AUTH	Lista relatórios
GET	/api/relatorios/{id}	AUTH	Detalha relatório
POST	/api/relatorios	ADMIN	Cria relatório (crise fechada, 1 por crise)
GET	/api/usuarios , /{id}	ADMIN	Lista / detalha usuários
POST/PUT/DELETE	/api/usuarios , /{id}	ADMIN	Cria / edita / desativa usuário
GET	/api/auditoria	ADMIN	Logs com filtros <code>?entidade&acao&usuario</code>
GET	/api/auditoria/facets	ADMIN	Valores distintos para filtros
GET	/api/{ref} , /{id}	AUTH	Lista / detalha dados de referência
POST/PUT/DELETE	/api/{ref} , /{id}	ADMIN	CRUD dos dados de referência

`{ref}` ∈ departamentos , tipos-crise , instituicoes , campi , cenarios . O `DELETE` é *soft-delete* (marca `active=false`).

8. Segurança, autenticação e auditoria

Autenticação por JWT em cookie HttpOnly

O token **não** trafega no header `Authorization` : o Quarkus é configurado para lê-lo de um cookie chamado `jwt` . Isso protege contra XSS (o JS do browser não acessa o token).

```
# application.properties
mp.jwt.verify.issuer=https://crisis-management/issuer
mp.jwt.verify.publickey.location=publicKey.pem      # verifica assinatura
smallrye.jwt.sign.key.location=privateKey.pem      # assina no login
mp.jwt.token.header=Cookie
mp.jwt.token.cookie=jwt
```

FLUXO DE LOGIN

1. `AuthService.login()` busca o usuário por e-mail e confere `active` + `BcryptUtil.matches()` .
2. Emite JWT (RS256) com `subject =id`, claims `name / email` , `groups` =perfil e expiração de **1 hora**.
3. `AuthController` devolve o token em cookie via `AuthCookieFactory` : `HttpOnly` , `SameSite=LAX` , `path=/` , `maxAge=3600` .
4. O logout devolve o mesmo cookie com `maxAge=0` (expira).

```
return Jwt.issuer(issuer)
    .subject(user.id.toString())
    .claim("name", user.name).claim("email", user.email)
    .groups(user.perfil.name()) // "ADMIN" ou "COMUM" → role
    .expiresIn(TOKEN_EXPIRATION_SECONDS) // 3600s
    .sign();
```

Autorização

- `@Authenticated` — exige sessão válida (qualquer perfil).
- `@RolesAllowed("ADMIN")` — restringe a administradores; a role vem do claim `groups` .
- `@RolesAllowed({"ADMIN", "COMUM"})` — ambos os perfis (ex.: registrar ação).
- Regra de negócio adicional no service: usuário `COMUM` só registra ação em crise do próprio departamento (senão `403 Forbidden`).

Auditoria automática — `AuditoriaLogFilter`

Um `ContainerResponseFilter` (JAX-RS) intercepta **toda resposta** e, para métodos de modificação (POST/PUT/PATCH/DELETE) feitos por usuário autenticado, grava um log com quem (subject/nome do JWT), a ação (CRIAR/ATUALIZAR/EXCLUIR), o método, a entidade derivada do path, o recurso e o status HTTP da resposta. Requisições anônimas e chamadas a `auth / auditoria` são ignoradas. Falhas na auditoria nunca interrompem a requisição original.

O registro roda em transação separada (`@Transactional(REQUIRES_NEW)`) porque o filtro executa após a transação da requisição original já ter sido encerrada.

Tratamento de erros

`InvalidCredentialsException` + `InvalidCredentialsExceptionMapper` convertem falha de login em `401` com corpo JSON (`LoginResponse`). Nos services, erros de regra usam exceções JAX-RS padrão: `NotFoundException` (404), `BadRequestException` (400), `ForbiddenException` (403).

CORS

Habilitado para o dev server Angular: origem `http://localhost:4200` , métodos GET/POST/PUT/DELETE/OPTIONS e `allow-credentials=true` (necessário para o cookie cruzar as origens).

9. Regras de negócio principais

Regra	Onde	Comportamento
Ciclo de vida da crise	CriseService	Nasce <code>ABERTA</code> ; avança <code>ABERTA</code> → <code>EM_ANDAMENTO</code> → <code>RESOLVIDA</code> → <code>ENCERRADA</code> . Crise <code>ENCERRADA</code> não muda mais de status.
Ação por departamento	CriseService.addAcao	Perfil <code>COMUM</code> só registra ação em crise do seu <code>departamentoId</code> (senão 403).
Relatório único e tardio	RelatorioCriseService	Só para crise <code>RESOLVIDA/ENCERRADA</code> ; máx. 1 por crise (constraint <code>UNIQUE</code> + checagem).
E-mail único	UsuarioService	Não permite dois usuários com o mesmo e-mail (criação e edição).
Senha na edição	UsuarioService	Senha vazia/nula mantém a atual; preenchida gera novo hash <code>BCrypt</code> .
Auto-proteção	UsuarioService.deactivate	Usuário não pode desativar a própria conta.
Soft-delete	Services de referência	<code>DELETE</code> marca <code>active=false</code> , preservando histórico.

10. Frontend — arquitetura Angular

Aplicação **100% standalone** (sem `NgModule`). Bootstrap em `main.ts` com `bootstrapApplication(App, appConfig)`; o componente raiz `App` só renderiza um `<router-outlet>`.

PROVIDERS GLOBAIS — APP.CONFIG.TS

```
providers: [  
  provideBrowserGlobalErrorListeners(),  
  provideZoneChangeDetection({ eventCoalescing: true }),  
  provideRouter(routes),  
  provideHttpClient(withInterceptors([credentialsInterceptor])),  
]
```

Estado e estilo

- **Signals** para estado de componente (`loading` , `error` , dados) e da sessão (`currentUser` , `isAuthenticated` = `computed( ... )`).
- **SCSS design system próprio** em `styles.scss` : tokens em `:root` , paleta verde institucional (`--primary:#2f9e41`), classes utilitárias (`btn` , `badge` , `role-chip`), ícones SVG inline (estilo Lucide). Sem Material/Bootstrap.
- `API_BASE_URL` = `'http://localhost:8080'` centraliza a base da API.

11. Frontend — rotas, guards e interceptors

Todas as features usam `loadComponent` (lazy). A casca `MainLayoutComponent` agrupa as rotas autenticadas sob `authGuard` ; só `/auditoria` tem guard extra `adminGuard` .

Rota	Componente	Guard
<code>/login</code>	<code>LoginComponent</code>	—
<code>/dashboard</code>	<code>DashboardComponent</code>	<code>authGuard</code>
<code>/crises</code>	<code>CriseListComponent</code>	<code>authGuard</code>
<code>/crises/nova</code>	<code>CriseFormComponent</code>	<code>authGuard</code>
<code>/crises/:id</code>	<code>CriseDetailComponent</code>	<code>authGuard</code>
<code>/relatorios</code> · <code>/relatorios/novo</code>	Relatorio List/Form	<code>authGuard</code>
<code>/departamentos</code> · <code>/tipos-crise</code> · <code>/instituicoes</code> · <code>/campi</code> · <code>/cenarios</code>	* <code>ListComponent</code>	<code>authGuard</code>
<code>/auditoria</code>	<code>AuditorialListComponent</code>	<code>authGuard</code> + <code>adminGuard</code>
**	—	redirect → <code>/login</code>

Guards (revalidam no backend)

- `authGuard` : chama `GET /auth/me` ; sucesso → libera, erro → redireciona a `/login` . A sessão é revalidada no servidor a cada navegação (não confia em flag local).
- `adminGuard` : idem, mas exige `perfil ≡ 'ADMIN'` ; autenticado sem permissão vai para `/dashboard` .

Interceptor de credenciais

```
export const credentialsInterceptor: HttpInterceptorFn = (req, next) =>
  next(req.clone({ withCredentials: true })); // envia o cookie jwt cross-origin
```

Diretiva de papel — `*appRole`

Diretiva estrutural que renderiza o elemento só se o perfil do usuário estiver na lista permitida (`*appRole="['ADMIN']"`). Usa `input.required()` + `effect()` e reage automaticamente a login/logout via o signal `currentUser` . Governa a *visibilidade* do menu (a proteção real de rota fica nos guards + backend).

12. Frontend — funcionalidades (features)

Crises (núcleo)

Serviço com todos os endpoints: `listAll`, `findById`, `create`, `changeStatus`, `listAcoes`, `createAcao`.

- **Lista** — carrega todas as crises no signal `crises`, com links para detalhe/criação.
- **Formulário (nova)** — a tela mais rica: carrega em paralelo os 4 selects (tipos, campi, cenários, departamentos) e envia `CriseRequest`. É restrita a ADMIN no menu.
- **Detalhe** — mostra a crise e suas ações; implementa a máquina de estados (`fluxo` : ABERTA→EM_ANDAMENTO→RESOLVIDA→ENCERRADA), expõe `proximoStatus` / `avancarStatus()` e só mostra o formulário de nova ação quando o status é ABERTA ou EM_ANDAMENTO.

Dashboard

Página única (landing pós-login). Consome `GET /api/dashboard/resumo` → `DashboardResumo { totalCrises, crisesAbertas, crisesEmAndamento, crisesResolvidas, crisesCriticas }`, exibindo KPIs que linkam para as crises.

Relatórios

`list` + `form`. No formulário, o select de crises é filtrado para exibir **apenas crises RESOLVIDA/ENCERRADA** (`crisesElegiveis`), espelhando a regra do backend.

Auditoria (só ADMIN)

Lista com filtros por entidade, ação e usuário (dropdowns vêm de `/facets`), badges coloridos por ação (CRIAR/ATUALIZAR/EXCLUIR) e por faixa de status HTTP, com `DatePipe` para a data/hora.

Dados de referência

Departamentos, Tipos de Crise, Instituições, Campi e Cenários. Os *serviços* expõem CRUD completo (incl. `update` e `deactivate`), porém, no estado atual, os *componentes de lista* apenas chamam `listAll()` — funcionando como listagens somente-leitura (não há rotas `/nova` ou `/:id` para eles). Apenas **crises** e **relatórios** têm telas de formulário.

13. Fluxos de ponta a ponta

Login

```
Login (Angular) --POST /auth/login--> AuthController
AuthService valida BCrypt + emite JWT
<--Set-Cookie: jwt (HttpOnly)-- navega para /dashboard
Toda navegação seguinte: authGuard --GET /auth/me--> valida sessão
```

Ciclo completo de uma crise

```
1. ADMIN cria crise      POST /api/crises      (status ABERTA)
2. Equipe registra ações POST /api/crises/{id}/acoes (COMUM: só seu depto.)
3. ADMIN avança status   PATCH /api/crises/{id}/status
    ABERTA → EM_ANDAMENTO → RESOLVIDA → ENCERRADA
4. ADMIN emite relatório POST /api/relatorios      (só RESOLVIDA/ENCERRADA)
* Cada POST/PUT/PATCH/DELETE gera 1 linha em auditoria_logs automaticamente.
```

14. Como executar o projeto

PRÉ-REQUISITOS

Java 21, Maven (wrapper incluso), Node.js + Angular CLI, e um PostgreSQL em `localhost:5433` com o banco `crisis_management` (usuário/senha `postgres`).

BACKEND (PORTA 8080)

```
# na raiz do projeto
./mvnw quarkus:dev      # modo dev com live reload
# Dev UI: http://localhost:8080/q/dev/
# Flyway aplica V1..V8 e o DataSeeder cria admin/usuario automaticamente
```

FRONTEND (PORTA 4200)

```
cd frontend
npm install
npm start      # ng serve → http://localhost:4200
```

CREDENCIAIS SEMEADAS

Perfil	E-mail	Senha
ADMIN	admin@crisis.local	Admin@123
COMUM	usuario@crisis.local	Comum@123